The Bridge School · TEAM CHALLENGE

Hundir la Flota

Guía completa de flujo de trabajo en equipo

Git · GitHub · PRs · Documentación · Presentación

FASE 1	FASE 2	FASE 3	FASE 4
Setup inicial	**Desarrollo**	**Integración**	**Presentación**

Esta guía acompaña el Team Challenge. Cada fase tiene pasos numerados y comandos listos para ejecutar. Seguid el orden — cada fase depende de la anterior.

FASE 1 DE 4

Setup inicial

Repositorio · Protección de ramas · Estructura del proyecto · Distribución de trabajo

¿Por qué esta fase es crítica?

Todo lo que construyáis depende de tener una base sólida. Un repositorio mal configurado al principio genera conflictos, pérdida de trabajo y commits sobre main que son muy difíciles de revertir. Dedicad 20 minutos aquí y ahorraréis horas después.

1

Crear el repositorio en GitHub

Una persona del equipo (el Scrum Master) se encarga de esto. El resto esperan hasta el paso 3.

- Ir a github.com → botón verde 'New repository'
- Nombre sugerido: hundir-la-flota
- Visibility: Public (para que el profesor pueda verlo)
- Marcar: Add a README file
- Marcar: Add .gitignore → plantilla Python
- NO marcar ninguna licencia

Una vez creado, copiad la URL del repo (botón verde 'Code' → HTTPS).

2 Añadir colaboradores

Todos los integrantes deben tener acceso de escritura antes de clonar.

1. Settings → Collaborators → Add people
2. Buscar por usuario de GitHub de cada compañero
3. Cada compañero acepta el email de invitación
4. Verificar: todos aparecen con rol 'Write'

3 Proteger la rama main

Esto es lo que hace que trabajéis como un equipo de verdad. Sin esta protección cualquiera puede subir código roto directamente.

5. Settings → Branches → Add branch ruleset
6. Enforcement status: Active
7. Target branches: Default branch

Activar las siguientes opciones:

- Require a pull request before merging
- Require approvals: 1 (un compañero debe aprobar antes de mergear)
- Dismiss stale reviews (si se añaden commits, la aprobación se resetea)
- Block force pushes

Resultado esperado: si alguien intenta hacer 'git push origin main' directamente, GitHub lo rechazará con error. Esto es correcto y es el comportamiento deseado.

4 Clonar el repositorio

Ahora cada integrante clona el repo en su máquina. Hacedlo todos a la vez.

git clone https://github.com/usuario/hundir-la-flota.git

Distribución de módulos y ramas

Cada integrante tiene un módulo principal. Con grupos de 3, el integrante 3 cubre funciones.py y apoya en main.py. Con grupos de 4, hay un rol de QA/integración.

Rol	Módulo	Nombre de rama	Responsabilidad principal
Scrum Master	variables.py	`feature/variables-[nombre]`	Constantes, estructura base, coordinar el equipo
Dev 1	clases.py	`feature/clases-[nombre]`	Clase Tablero: constructor, coloca_barco, recibe_disparo

Dev 2 / QA	funciones.py	`feature/funciones-[nombre]`	Funciones auxiliares, helpers del juego
Dev 3 (opt.)	main.py	`feature/main-[nombre]`	Bucle principal, integración final de módulos

5 Crear y publicar la rama de cada uno

Cada integrante ejecuta esto en su máquina, con su nombre en la rama.

```
# Asegúrate de estar en main y actualizado
git checkout main
git pull origin main

# Crear la rama y publicarla inmediatamente en remoto
git checkout -b feature/variables-ana
git push -u origin feature/variables-ana

# Verificar que la rama remota existe
git branch -a
```

Regla de oro: vuestra rama es vuestro espacio. No trabajéis nunca en la rama de un compañero sin avisar. Si necesitáis algo de otra rama, usad git cherry-pick o abrid un PR.

6 Crear los Issues en GitHub

Los Issues son las tareas del proyecto. Crearlos antes de empezar a codear os obliga a pensar qué hay que hacer y permite vincular cada PR a una tarea.

8. En el repo: pestaña Issues → New Issue
9. Crear uno por módulo + uno para README + uno para demo

#	Título del Issue	Asignado a	Descripción mínima
#1	Definir variables.py y constantes	Scrum Master	BOARD_SIZE, SHIPS dict, símbolos, mensajes
#2	Implementar clase Tablero	Dev 1	Constructor, place_ships(), receive_shot(), display()
#3	Implementar funciones.py	Dev 2	Funciones auxiliares del flujo del juego
#4	Implementar main.py	Dev 3 / todos	Bucle while True, integración final
#5	Escribir README.md	Scrum Master	Instrucciones, estructura, simbología del tablero
#6	Demo y presentación	Todos	Juego funcional, preparar demo de 10 min

10. Asignar cada Issue a su responsable (Assignees)

11. Crear un Milestone llamado 'Sprint 5' y asignar todos los Issues
12. En Projects: crear un tablero Kanban con columnas To Do / In Progress / In Review / Done

FASE 2 DE 4

Desarrollo

Conventional Commits · Docstrings · Sincronización diaria · Code Review

El orden de desarrollo importa

variables.py es la base de todo. No podéis implementar clases.py sin saber las constantes, ni funciones.py sin la clase Tablero. Respetad este orden de desarrollo:

1	`variables.py`	→	Sin dependencias. Empezar aquí. Compartir con el equipo en cuanto esté.
2	`clases.py`	→	Importa variables.py. Clase Tablero completa antes de empezar funciones.py.
3	`funciones.py`	→	Puede ir en paralelo con el final de clases.py. Importa clases y variables.
4	`main.py`	→	Solo cuando los tres módulos anteriores funcionan. Integra todo.

7

Conventional Commits — el formato obligatorio

Un buen historial de commits es la diferencia entre un proyecto profesional y un caos. Usad siempre este formato:

```
tipo: descripción corta en presente e imperativo

# TIPOS válidos:
feat:     nueva funcionalidad
fix:      corrección de bug
refactor: reorganización sin cambio de comportamiento
docs:     documentación (docstrings, README)
test:     pruebas manuales o automatizadas
chore:    cambios de configuración (.gitignore, etc.)
```

Ejemplos correctos:

```
feat: añadir clase Tablero con constructor básico
feat: implementar método place_ships con orientación aleatoria
fix: corregir desbordamiento de barco en borde derecho del tablero
refactor: extraer validación de coordenadas a función auxiliar
docs: añadir docstrings a todos los métodos de Tablero
chore: actualizar .gitignore para excluir __pycache__
```

Mal commit	Buen commit
✗ arreglado	✓ feat: añadir método receive_shot
✗ cambios	✓ fix: corregir índice fuera de rango
✗ wip	✓ docs: docstring en place_ships
✗ commit	✓ refactor: simplificar bucle de validación
✗ funciona	
✗ final final v2	

8 Docstrings — documentar mientras se desarrolla

No lo dejéis para el final. Documentad cada función y clase mientras la escribís. El formato estándar en Python es Google Style:

```
class Tablero:
    """
    Representa el tablero de juego de un jugador.

    Attributes:
        player_id (str): Identificador del jugador.
        size (int): Dimensión del tablero (size x size).
        board (np.ndarray): Array con barcos e impactos.
        tracking (np.ndarray): Array de seguimiento de disparos al rival.
    """

    def receive_shot(self, x: int, y: int) -> bool:
        """
        Procesa un disparo en la coordenada (x, y).

        Args:
            x (int): Fila del disparo (0-9).
            y (int): Columna del disparo (0-9).

        Returns:
            bool: True si impacta en barco, False si es agua.

        Raises:
            ValueError: Si las coordenadas están fuera del tablero.
        """
```

9 Flujo de trabajo diario

Cada vez que os pongáis a trabajar, seguid este ciclo. No saltéis ningún paso.

13. Sincronizar main (por si un compañero mergeó algo):

```
git checkout main
git pull origin main
```

14. Volver a vuestra rama y hacer rebase:

```
git checkout feature/clases-maria
```

```
git rebase main
# Si hay conflictos, resolverlos y:
git add archivo_con_conflicto.py
git rebase --continue
```

15. Desarrollar, añadir y hacer commit:

```
# Trabajar en el código...

git add clases.py
git commit -m 'feat: implementar método place_ships con orientación N/S/E/O'
```

16. Subir los cambios al remoto:

```
git push origin feature/clases-maria
# Si hiciste rebase, puede que necesites:
git push --force-with-lease origin feature/clases-maria
```

¿Qué es --force-with-lease? Es un push forzado 'seguro': sobreescribe tu rama remota SOLO si nadie más ha subido cambios a ella mientras tanto. Nunca uséis --force a secas.

FASE 3 DE 4

Integración

Pull Requests · Code Review · Merge · Resolución de conflictos

El ciclo completo de un Pull Request

Este es el flujo que replica lo que haréis en cualquier empresa. Cada cambio que entra en main debe pasar por este ciclo sin excepción.

10

Abrir el Pull Request

Cuando vuestro módulo esté listo para revisión (no tiene que ser perfecto, sí tiene que funcionar), abrid el PR.

17. En GitHub: pestaña Pull Requests → New Pull Request
18. Base: main ← Compare: feature/clases-maria
19. Título: seguir el mismo formato que los commits
20. Descripción: usar esta plantilla:

```
## ¿Qué hace este PR?
Implementa la clase Tablero con constructor, place_ships() y receive_shot().

## Cambios
- clases.py: clase Tablero completa con los 3 métodos requeridos
- variables.py: añadidas constantes HIT y MISS que faltaban
```

```
## Cómo probarlo
python -c "from clases import Tablero; t = Tablero('player'); t.place_ships();
print(t)"

## Checklist
- [x] El código funciona sin errores
- [x] Docstrings en todos los métodos
- [x] Sin print() de debug
- [x] Sin valores hardcoded
- [x] He hecho rebase de main antes de abrir el PR

Closes #2
```

21. Assignees: vosotros mismos (el autor)
22. Reviewers: asignad a un compañero distinto al autor
23. Vincular el Issue: en la barra lateral, 'Development' → seleccionar el issue

11 Code Review — cómo revisar y cómo recibir la revisión

El reviewer es el guardián de la calidad. Su trabajo no es aprobar todo — es encontrar problemas antes de que entren en main.

¿Qué mirar al revisar?

- ¿El código hace lo que dice el PR?
- ¿Los nombres de variables y funciones son descriptivos?
- ¿Hay valores hardcoded que deberían estar en variables.py?
- ¿Hay print() de debug olvidados?
- ¿Las funciones tienen docstrings?
- ¿Hay casos edge no manejados? (coordenadas fuera de rango, tablero lleno, etc.)
- ¿El código es legible? ¿Hay partes confusas sin comentario?

Cómo comentar en GitHub

En la pestaña 'Files changed', haced clic en el número de línea que queréis comentar. Escribid comentarios específicos y constructivos:

Comentario vago (mal)	Comentario específico (bien)
"Esto no me gusta" *"Cámbialo"* *"No funciona"*	*"Este 10 debería ser BOARD_SIZE de variables.py"* *"¿Qué pasa si x < 0? Falta validar el rango"*

Cómo responder a una revisión

- Leed cada comentario con cuidado y sin ponerse a la defensiva
- Responded a cada comentario: o hacéis el cambio o explicáis por qué no
- Cuando hayáis corregido todo, subid los commits y pedid re-revisión
- Mínimo una ronda de revisión antes de mergear

12 Merge y limpieza de rama

Una vez el reviewer aprueba el PR, el autor hace el merge. Usad siempre Squash and Merge para mantener el historial de main limpio.

24. En el PR aprobado: botón 'Squash and Merge' → Confirm
25. Borrar la rama remota (GitHub lo propone automáticamente)
26. Actualizar main local y limpiar la rama local:

```
git checkout main
git pull origin main

# Eliminar la rama local (ya mergeada)
git branch -d feature/clases-maria

# Los demás sincronizan también:
git pull origin main
git rebase main  # desde su rama activa
```

¿Por qué Squash and Merge? Durante el desarrollo hacéis muchos commits pequeños (wip, prueba, arreglado). Squash los aplana en uno solo con un mensaje limpio en main. El historial de main queda legible para cualquiera que lo vea después.

13 Resolver conflictos de merge

Los conflictos ocurren cuando dos personas modifican las mismas líneas. No son un error — son normales en trabajo en equipo. Aquí cómo resolverlos:

```
# Al hacer rebase aparece un conflicto:
# CONFLICT (content): Merge conflict in variables.py

# Abrir el archivo, el conflicto se ve así:
<<<<<<< HEAD (vuestro cambio)
BOARD_SIZE = 10
=======
BOARD_SIZE = 12
>>>>>>> main (cambio de main)

# Decidid qué versión es correcta, editad el archivo,
# eliminad las marcas <<< === >>> y guardad.

git add variables.py
git rebase --continue
```

VS Code tiene una interfaz visual para conflictos: os muestra 'Accept Current', 'Accept Incoming' o 'Accept Both'. Es más cómodo que editar el texto a mano.

FASE 4 DE 4

Presentación

Demo en vivo · Estructura de 10 minutos · Qué enseñar del repo

La presentación es parte del proyecto

La demo no es un añadido al final — es la prueba de que el juego funciona. Tened el código listo al menos un día antes para poder ensayar el timing.

14 Estructura de los 10 minutos

Tiempo	Bloque	Contenido
0:00 – 1:30	Intro del equipo	Presentaros, decir qué ha hecho cada uno. 30 seg máximo por persona.
1:30 – 4:00	Tour del repo	Estructura de archivos, ramas, historial de commits, PRs mergeados, Project Board.
4:00 – 5:30	Código clave	Mostrar la clase Tablero: place_ships() y receive_shot(). Explicar la lógica, no leer el código.
5:30 – 8:30	Demo en vivo	Partida completa desde el terminal. Mostrar ambos tableros, un impacto, un fallo, hasta el final o durante 3 minutos.
8:30 – 10:00	Retrospectiva	¿Qué fue bien? ¿Qué fue difícil? ¿Qué haríais diferente? Una respuesta honesta cada uno.

15 Checklist del repositorio antes de presentar

Revisad esto el día anterior, no 5 minutos antes:

✓	README.md presente con instrucciones de ejecución y simbología del tablero
✓	Todos los archivos (.py) tienen docstrings en funciones y clases
✓	Sin print() de debug en el código final
✓	Sin valores hardcoded: todo en variables.py
✓	Al menos 1 PR mergeado por integrante en el historial
✓	Issues cerrados o vinculados a sus PRs
✓	GitHub Project Board actualizado (todos los Issues en Done)
✓	python main.py funciona desde cero en una máquina limpia
✓	El juego llega hasta condición de victoria o derrota
✓	Ensayada la demo: sabéis qué coordenadas vais a introducir

16

Consejos para la demo en vivo

- Tened el terminal abierto con el directorio correcto antes de empezar
- Ejecutad python main.py una vez antes de presentar para verificar que funciona
- Tened preparadas las coordenadas que vais a introducir: alternad hits y misses para que sea visualmente interesante
- Si el juego falla en directo: no os agobiéis, explicad qué estaba pasando. La capacidad de depurar en directo también se valora
- Zoom en el terminal si lo proyectáis: font-size grande, fondo oscuro
- El que habla no es el que teclea. Dividid los roles

Consejo final: la presentación no es solo sobre si el juego funciona. Es sobre demostrar que sabéis lo que habéis construido y que habéis trabajado como un equipo real. El proceso (Git, PRs, reviews) es tan importante como el resultado.

Referencia rápida — comandos esenciales

Situación	Comando
Inicio del día	`git checkout main && git pull origin main`
Volver a mi rama y sincronizar	`git checkout feature/mi-rama && git rebase main`
Ver en qué rama estoy	`git branch`
Ver estado de mis cambios	`git status`
Añadir y commitear	`git add archivo.py && git commit -m 'feat: ...'`
Subir al remoto	`git push origin feature/mi-rama`
Subir tras un rebase	`git push --force-with-lease origin feature/mi-rama`
Ver historial compacto	`git log --oneline --graph`
Descartar cambios locales	`git checkout -- archivo.py`
Ver diferencias antes de commitear	`git diff`

¡Buena suerte con el desafío!

Cualquier duda, preguntad. El proceso importa tanto como el código.